

LINQ Group Join in C# with Examples

In this article, I will discuss **How to Implement a LINQ Group Join in C#** using Method and Query Syntax with Examples. Please read our previous article discussing [How to Join Multiple Data Sources in LINQ](#) with Examples.

What is LINQ Group Join?

The Group Join method is used to perform a group join operation between two sequences (collections) based on matching keys extracted from the elements of each sequence. The GroupJoin method is a way to combine elements from two sources based on key equality and, for each matching element in the first (outer) sequence, groups all matching elements in the second (inner) sequence. The result is a hierarchical structure where each element from the outer sequence is paired with a collection of matched elements from the inner sequence. This operation is particularly useful when you have a one-to-many relationship between the elements in the sequences.

It is a combination of the “join” and “group by” operations found in SQL. Here’s a basic outline of how GroupJoin works in LINQ:

- **Collections:** You need two collections to perform a Group Join. These can be arrays, lists, or any enumerable collections.
- **Key Selector:** For each of the two collections, you define a key selector function. This function specifies the key used to match elements from both collections.
- **Result Selector:** After matching elements based on the key, a result selector function defines how to shape the output. This function typically creates a new type that includes the key and the grouped results as a collection.

Syntax to Use LINQ Group Join:

```
var groupJoinQuery = outerSequence.GroupJoin(  
    innerSequence,  
    outerKeySelector,  
    innerKeySelector,  
    (outerElement, innerElements) => new {  
        Outer = outerElement,  
        Inners = innerElements  
    });
```

Here is the explanation of the above syntax:

- **outerSequence:** This represents the first (outer) sequence to join. It’s one of the two collections you’re working with.
- **innerSequence:** This represents the second (inner) sequence to join. It’s the other collection you’re working with.
- **outerKeySelector:** A function to extract the key from each element of the outer sequence. This key is used to match elements from the inner sequence.
- **innerKeySelector:** A function to extract the key from each element of the inner sequence. This key is matched against the key from the outer sequence elements.

- **(outerElement, innerElements) => new { Outer = outerElement, Innners = innerElements }**: This is a result selector function that specifies what to do with each matched pair of elements. For each element in the outer sequence, it groups all matching elements from the inner sequence. The result is an anonymous type containing the outer element and an enumerable of matched inner elements. Outer is the element from the outer sequence, and Innners is the collection of matched elements from the inner sequence.

Examples to Understand LINQ Group Join in C#:

The GroupJoin operation is beneficial when you want to maintain the relationship between the elements of the primary collection and the elements of the secondary collection that share a common key. For example, you might have a list of departments and a list of employees. Using GroupJoin, you can associate each department with its list of employees in a single hierarchical structure.

Let us understand how to Implement LINQ Group Join with examples using C# language. For this, we will use the following two model classes: Employee and Department. So, create a class file named **Employee.cs** and copy and paste the following code. This class has three properties: ID, Name, and DepartmentId. We have also created one method to return a collection of Employees, which will be our first data source for the Group Join.

```
using System.Collections.Generic;
namespace LINQGroupJoin
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int DepartmentId { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", DepartmentId = 10},
                new Employee { ID = 2, Name = "Priyanka", DepartmentId = 20},
                new Employee { ID = 3, Name = "Anurag", DepartmentId = 30},
                new Employee { ID = 4, Name = "Pranaya", DepartmentId = 30},
                new Employee { ID = 5, Name = "Hina", DepartmentId = 20},
                new Employee { ID = 6, Name = "Sambit", DepartmentId = 10},
                new Employee { ID = 7, Name = "Happy", DepartmentId = 10},
                new Employee { ID = 8, Name = "Tarun", DepartmentId = 0},
                new Employee { ID = 9, Name = "Santosh", DepartmentId = 10},
                new Employee { ID = 10, Name = "Raja", DepartmentId = 20},
                new Employee { ID = 11, Name = "Ramesh", DepartmentId = 30}
            };
        }
    }
}
```

Next, create another class file named **Department.cs** and copy and paste the following code. This class has 2 properties, i.e., ID and Name. We have also created one method to return a collection of departments, which will be our second data source for the Group Join.

```

using System.Collections.Generic;
namespace LINQGroupJoin
{
    public class Department
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Department> GetAllDepartments()
        {
            return new List<Department>()
            {
                new Department { ID = 10, Name = "IT"},
                new Department { ID = 20, Name = "HR"},
                new Department { ID = 30, Name = "Sales" },
            };
        }
    }
}

```

As you can see, we created the above Employee and Department classes with some simple properties. The common property is Department ID, i.e., the ID property in the Department class and DepartmentID property in the Employee class. Then, we create two simple methods to return the respective data sources. Further, if you notice, the employee with ID 8 does not have a department.

Example to Understand LINQ GroupJoin Method Using Method Syntax in C#

Now, our business requirement is to group the employees by department. So, the outer data source will be the department data source, and the inner data source will be the employee data source. We need to use the **LINQ GroupJoin Method** to group the employees by department. The following code snippet shows how to group the employees by department using the LINQ GroupJoin Method using Method Syntax.

```

Department.GetAllDepartments(). //Outer Data Source
GroupJoin(
    Employee.GetAllEmployees(), //Inner Data Source
    dept => dept.ID, //Outer Key Selector
    emp => emp.DepartmentId, //Inner key Selector
    (dept, emp) => new {dept, emp} //Result Selector
);

```

As you can see in the above code snippet, our Outer Data Source is the Department collection, and our Inner Data Source is the Employee collection. Here, we are accessing the Department collection using the dept variable and the Employees collection using the emp variable. Here, the Outer Key Selector is the ID property of the Department class, and the Inner Key Selector is the DepartmentId property of the Employee class. Finally, we are projecting the dept and emp to an Anonymous type. The complete example code is given below. The following example code is self-explained, so please go through the comment lines.

```

using System.Linq;
using System;
namespace LINQGroupJoin

```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Group Employees by Department using Method Syntax
            var GroupJoinMS = Department.GetAllDepartments(). //Outer Data Source
            Departments
                GroupJoin( //Performing Group Join with Inner Data Source
                    Employee.GetAllEmployees(), //Inner Data Source
                    dept => dept.ID, //Outer Key Selector i.e. the Common Property
                    emp => emp.DepartmentId, //Inner Key Selector i.e. the Common Property
                    (dept, emp) => new { dept, emp } //Projecting the Result into
                    Anonymous Type
                );
            //Printing the Result set
            //Outer Foreach is for Each department
            foreach (var item in GroupJoinMS)
            {
                Console.WriteLine("Department :" + item.dept.Name);
                //Inner Foreach loop for each employee of a Particular department
                foreach (var employee in item.emp)
                {
                    Console.WriteLine("    EmployeeID : " + employee.ID + " , Name : " +
                    employee.Name);
                }
            }
            Console.ReadLine();
        }
    }
}

```

Output:

```

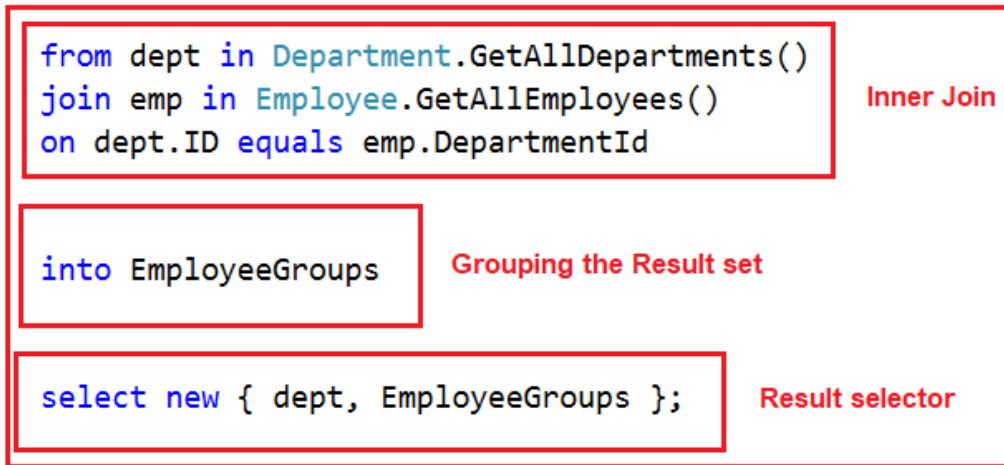
Department :IT
EmployeeID : 1 , Name : Preety
EmployeeID : 6 , Name : Sambit
EmployeeID : 7 , Name : Happy
EmployeeID : 9 , Name : Santosh
Department :HR
EmployeeID : 2 , Name : Priyanka
EmployeeID : 5 , Name : Hina
EmployeeID : 10 , Name : Raja
Department :Sales
EmployeeID : 3 , Name : Anurag
EmployeeID : 4 , Name : Pranaya
EmployeeID : 11 , Name : Ramesh

```

As you can see, the employee with ID 8 does not display here. This is because the employee with ID 8 does not belong to any department.

Example to Understand LINQ GroupJoin Using Query Syntax in C#

In LINQ Query Syntax, no such Group Join operator is available. Here, we need to use the LINQ Inner Join and the “into” operator. For a better understanding, please look at the following code snippet. Here, we have divided the code snippet into three sections for a better understanding. In the first section, we perform the LINQ Inner Join Operation between the Department and Employee Data Sources. In the second section, we project the result of the Inner Join into a variable called EmployeeGroups using the “into” operator. In the final section, we project the final result set as dept and EmployeeGroups.



It will group employees by department. That means one department can have multiple employees, or you can say one to many relationships between Department and Employees. The complete example code is given below. The following example code is self-explained, so please go through the comment lines. Here, we are using LINQ Query Syntax to implement Group Join.

```
using System.Linq;
using System;
namespace LINQGroupJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Group Employees by Department using Query Syntax
            var GroupJoinQS = from dept in Department.GetAllDepartments() //Outer
Source i.e. Departments
                                join emp in Employee.GetAllEmployees() //Joining
Data Source i.e. Employees
                                on dept.ID equals emp.DepartmentId //Joining Cor
                                into EmployeeGroups //Projecting the Joining Res
EmployeeGroups
                                //Final Result include each department and the
corresponding employees
                                select new { dept, EmployeeGroups };
            //Printing the Result set
        }
    }
}
```

```

//Outer Foreach is for Each department
foreach (var item in GroupJoinQS)
{
    Console.WriteLine("Department :" + item.dept.Name);
    //Inner Foreach loop for each employee of a Particular department
    foreach (var employee in item.EmployeeGroups)
    {
        Console.WriteLine("  EmployeeID : " + employee.ID + " , Name : " + employee.Name);
    }
    Console.ReadLine();
}
}
}

```

Run the application and print the same output as the previous example, as shown in the image below. The employee with ID 8 does not display here. This is because the employee with ID 8 does not belong to any department.

```

Department :IT
  EmployeeID : 1 , Name : Preety
  EmployeeID : 6 , Name : Sambit
  EmployeeID : 7 , Name : Happy
  EmployeeID : 9 , Name : Santosh
Department :HR
  EmployeeID : 2 , Name : Priyanka
  EmployeeID : 5 , Name : Hina
  EmployeeID : 10 , Name : Raja
Department :Sales
  EmployeeID : 3 , Name : Anurag
  EmployeeID : 4 , Name : Pranaya
  EmployeeID : 11 , Name : Ramesh

```

Projecting the Result to a Named Type:

So far, in the examples we have discussed, we have projected the result to an anonymous type. Can we Project the Result to a Named type? Yes, it is also possible to project the result to a named type instead of an anonymous type. Let us see how we can do this. First, create a class file named **GroupEmployeeByDepartment.cs** with the required properties you want in the result set. We have created the class with the following two properties per our requirements.

```

using System.Collections.Generic;
namespace LINQGroupJoin
{
    class GroupEmployeeByDepartment
    {
        public Department Department { get; set; }
    }
}

```

```

        public List<Employee> Employees { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows. Here, you can see we are projecting the result to the above-created GroupEmployeeByDepartment type.

```

using System.Linq;
using System;
namespace LINQGroupJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Group Employees by Department using Method Syntax
            var GroupJoinMS = Department.GetAllDepartments(). //Outer Data Source
            Departments
                GroupJoin( //Performing Group Join with Inner Data Source
                    Employee.GetAllEmployees(), //Inner Data Source
                    dept => dept.ID, //Outer Key Selector i.e. the Common Property
                    emp => emp.DepartmentId, //Inner Key Selector i.e. the Common Property
                    (dept, emp) => new GroupEmployeeByDepartment
                    {
                        Department = dept,
                        Employees = emp.ToList()
                    }
                );
            //Group Employees by Department using Query Syntax
            var GroupJoinQS = from dept in Department.GetAllDepartments() //Outer Data Source i.e. Departments
                               join emp in Employee.GetAllEmployees() //Joining Inner Data Source i.e. Employees
                               on dept.ID equals emp.DepartmentId //Joining Condition
                               into EmployeeGroups //Projecting the Joining Result to EmployeeGroups
                               //Projecting the Result to a Named Type
                               select new GroupEmployeeByDepartment
                               {
                                   Department = dept,
                                   Employees = EmployeeGroups.ToList()
                               };
            foreach (var item in GroupJoinQS)
            {
                Console.WriteLine("Department :" + item.Department.Name);
                foreach (var employee in item.Employees)
                {
                    Console.WriteLine("    EmployeeID : " + employee.ID + " , Name : " + employee.Name);
                }
            }
        }
    }
}

```

```

    }
}
Console.ReadLine();
}
}
}

```

With the above changes in place, run the application code, and it will give you the following output.

```

Department :IT
EmployeeID : 1 , Name : Preety
EmployeeID : 6 , Name : Sambit
EmployeeID : 7 , Name : Happy
EmployeeID : 9 , Name : Santosh
Department :HR
EmployeeID : 2 , Name : Priyanka
EmployeeID : 5 , Name : Hina
EmployeeID : 10 , Name : Raja
Department :Sales
EmployeeID : 3 , Name : Anurag
EmployeeID : 4 , Name : Pranaya
EmployeeID : 11 , Name : Ramesh

```

Projecting to Anonymous Type with User-Defined Property Names in ResultSet:

It is also possible to specify user-defined names in the result set. For a better understanding, please have a look at the following example. In the below example, we are projecting the result to an anonymous type with user-defined property names. The following example is the same as the previous example, except we are projecting to an anonymous type.

```

using System.Linq;
using System;
namespace LINQGroupJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Group Employees by Department using Method Syntax
            var GroupJoinMS = Department.GetAllDepartments(). //Outer Data Source
            Departments
                GroupJoin( //Performing Group Join with Inner Data Source
                    Employee.GetAllEmployees(), //Inner Data Source
                    dept => dept.ID, //Outer Key Selector i.e. the Common Property
                    emp => emp.DepartmentId, //Inner Key Selector i.e. the Common Property
                    //Projecting the Result with User Defined Names
                    (dept, emp) => new
                    {

```



```

        Department = dept,
        Employees = emp.ToList()
    }
    );
    //Group Employees by Department using Query Syntax
    var GroupJoinQS = from dept in Department.GetAllDepartments() //Out
Source i.e. Departments
                        join emp in Employee.GetAllEmployees() //Joining
Data Source i.e. Employees
                        on dept.ID equals emp.DepartmentId //Joining Cor
into EmployeeGroups //Projecting the Joining Res
EmployeeGroups

                        //Projecting the Result with User Defined Names
                        select new
                        {
                            Department = dept,
                            Employees = EmployeeGroups.ToList()
                        };
    foreach (var item in GroupJoinQS)
    {
        Console.WriteLine("Department :" + item.Department.Name);
        foreach (var employee in item.Employees)
        {
            Console.WriteLine("  EmployeeID : " + employee.ID + " , Na
employee.Name);
        }
    }
    Console.ReadLine();
}
}
}
}

```

Output:

```

Department :IT
  EmployeeID : 1 , Name : Preety
  EmployeeID : 6 , Name : Sambit
  EmployeeID : 7 , Name : Happy
  EmployeeID : 9 , Name : Santosh
Department :HR
  EmployeeID : 2 , Name : Priyanka
  EmployeeID : 5 , Name : Hina
  EmployeeID : 10 , Name : Raja
Department :Sales
  EmployeeID : 3 , Name : Anurag
  EmployeeID : 4 , Name : Pranaya
  EmployeeID : 11 , Name : Ramesh

```

When Should We Use LINQ Group Join?

The LINQ GroupJoin method is particularly useful in scenarios where you want to create a one-to-many relationship between two sequences. Here are some common situations where GroupJoin is beneficial:

- **Creating Hierarchical Results:** Use Group Join when you want to organize the results in a parent-child relationship where a single parent element is associated with one or more child elements. It's particularly useful for creating nested data structures, for example, when you have a collection of departments and a collection of employees, and you want to list each department along with all of its employees.
- **Handling One-to-Many Relationships:** Group Join is ideal for dealing with one-to-many relationships in your data. For instance, if you're working with a database of customers and orders, and you want to retrieve each customer along with all their orders, Group Join can efficiently accomplish this by grouping the orders by the customer.
- **Data Transformation and Aggregation:** Group Join can be quite handy when you need to transform data from multiple sources and aggregate it somehow. It allows you to correlate elements from different collections and then perform operations like counting, summing, or averaging on the grouped elements. For example, the total sales per department can be calculated from a collection of sales records.
- **Combining Data from Multiple Sources:** Use Group Join when you need to combine data from multiple sources. This is common in scenarios where data in different collections or tables need to be viewed together, such as combining product information with inventory levels from separate sources.
- **Efficiency in Querying Large Datasets:** In scenarios where large datasets are involved, and complex queries involve grouping and joining, LINQ Group Join can be more efficient than traditional loops and manual collection manipulations. The .NET runtime can optimize LINQ queries, potentially offering better performance.

In the next article, I will discuss the [LINQ Left Outer Join in C#](#) with Examples. In this article, I explain the LINQ GroupJoin using **Method and Query Syntax** Examples in C#. I hope you enjoy this LINQ GroupJoin Method in C# with Examples article.